BDH-GPU for Searchless Chess

Training a Post-Transformer Architecture on Chess Position Evaluation

Antoni Czapski

Project: Deep Learning
Proposal Presentation

March 10, 2026

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Searchless Chess: The Problem

Task: Predict chess position quality from a static board state — no tree search allowed.

- **Input:** Board position (FEN notation)
- **Output:** Score $\in [-1, 1]$ (normalized Stockfish centipawn)
- **Formulation:** Supervised regression

No minimax. No alpha-beta. No MCTS. Pure neural intuition.

Prior Art

Model	Params	ELO
DeepMind	270M	2895
ViT-Med*	9.5M	1960
ViT-Small*	2.6M	1817
ResNet-XL*	24.7M	1719

*Grzyb, 2025

Key insight: ViTs achieve 10× better parameter efficiency than ResNets for chess.

My Angle: Why BDH?

Research question:
Can a post-transformer architecture (BDH-GPU) match or beat ViT on searchless chess?

Why BDH suits chess:

1. **Relational state** — piece interactions (pins, forks, batteries) stored as neuron-pair weights
2. **Iterative reasoning** — shared-weight layers applied K times, like "thinking longer"
3. **Sparse activations** — only relevant motifs fire, matching chess pattern recognition

BDH in One Sentence

A recurrent linear-attention model with Hebbian fast-state updates and weight-tied layers.

Novelty

No prior work has applied BDH to chess or any combinatorial game domain.

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Dataset: Lichess-Stockfish-Normalized

Source: HuggingFace (CC BY 4.0)
`huggingface.co/datasets/mateuszgrzyb/lichess-stockfish-normalized`

Property	Value
Total positions	316M
Format	Parquet (10 shards)
Deduplication	FEN-based
Key columns	`fen`, `cp`, `depth`
Original source	Lichess.org

I use: ∼50–100M subset
(compute-constrained to 100h A100)

Input Encoding

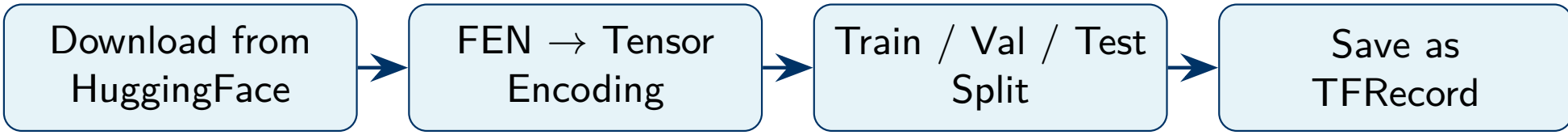
FEN → 8×8×12 tensor

- 64 squares × 12 channels
- Binary piece-type occupancy
- Always-white perspective (board mirrored for black)

Data Split

Train	90%
Validation	5%
Test	5%
Seed	2001

Data Pipeline



Reusable Components

- `fen_to_tensor()` — proven encoding from existing repo
- `mate_to_cp()` — mate → centipawn conversion
- TFRecord streaming for memory efficiency

Engineering Decisions

- **Deterministic** splits (seeded shuffle)
- **Config-driven** — all paths & params in YAML
- **Scripted** download — `datasets` library
- **No raw data** in Git

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Evaluation Strategy

Training Metric:

$$\mathcal{L} = \text{MSE}(\hat{y}, y) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Monitoring: MAE for interpretability

Gold Standard: ELO Estimation

1. Solve 1,200 Lichess puzzles (12 tiers, 100 per tier)
2. Accuracy per tier
3. Linear regression fit
4. ELO = rating at 50% accuracy

Why Puzzles Are Valid

Searchless models evaluate only the **immediate position** (depth = 1).

$\Rightarrow$ Puzzle difficulty $\equiv$ pattern recognition complexity

$\Rightarrow$ 50% threshold $\approx$ human-equivalent strength at that rating

Puzzle Dataset

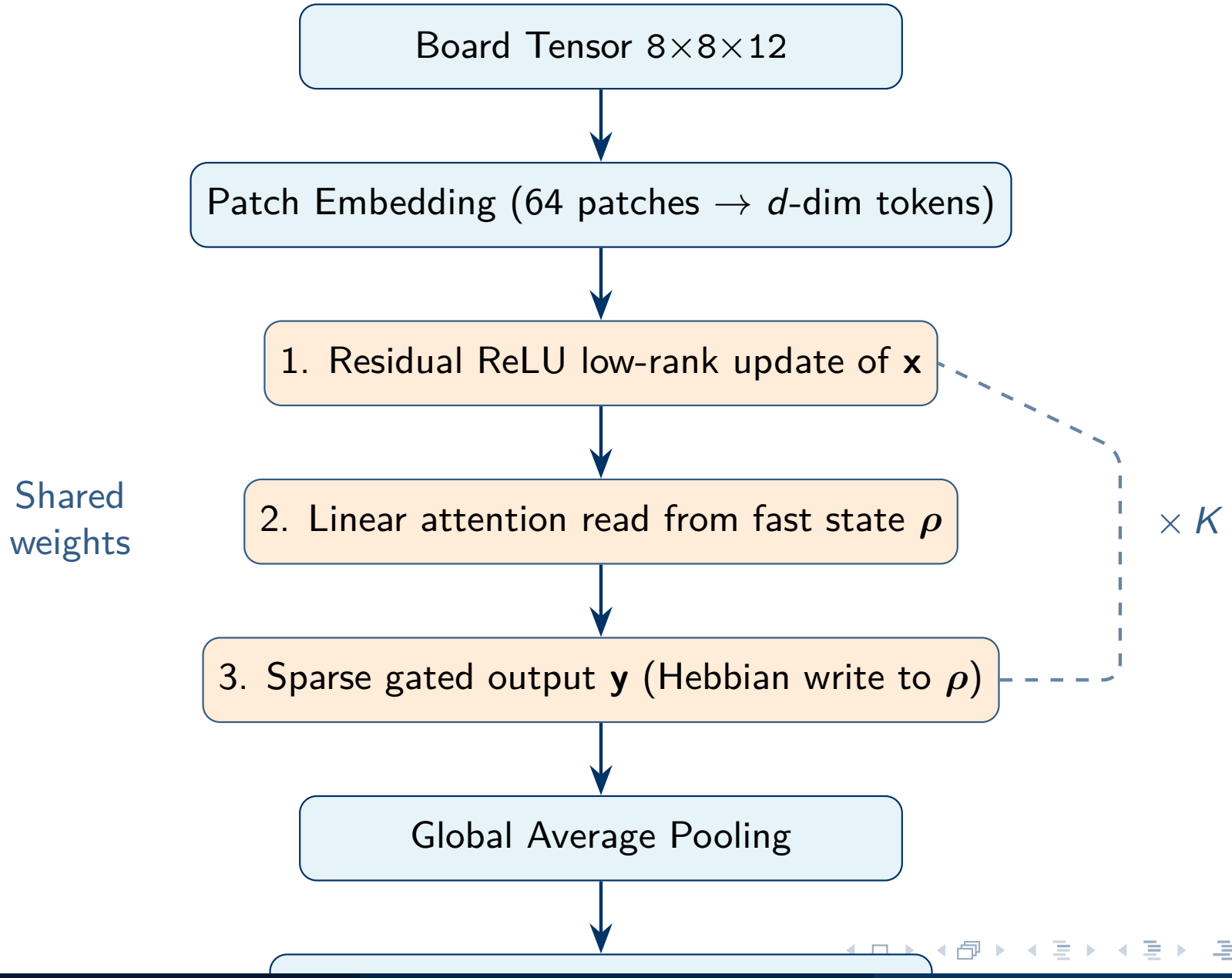
Already available:
`data/puzzles/test_puzzles.feather`

- Ratings: 399 – 3213 ELO
- Proven methodology (Grzyb, 2025)

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

BDH-GPU Architecture



Model Variants & Baselines

BDH-GPU Variants:

Variant	n	K	Params
BDH-Tiny	4096	4	∼1.6M
BDH-Small	8192	8	∼6.3M
BDH-Medium	16384	12	∼12.6M

n = neuron dimension, $d = 256$ (rank),
K = recurrent depth (shared weights)

Key Experiment:
Depth ablation — $K \in \{4, 8, 12\}$
Does more "thinking" help?

Baselines for Comparison:

Model	Params	ELO*
MLP	∼2M	∼900
ViT-Small	2.6M	1817

*Expected / known from prior work

Comparison Strategy

- **Param-matched:** BDH-Tiny vs ViT-Small (∼2.6M)
- **Scale:** BDH-Small/Medium
- **Ablation:** depth K effect

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Repository Structure

Directory	Purpose
`.github/workflows/`	CI/CD pipeline
`configs/`	YAML experim
`src/models/`	BDH, ViT, Res
`src/training/`	Unified training
`src/evaluation/`	Puzzle-based E
`src/data_preparation/`	FEN encoding
`tests/`	pytest test suit
`scripts/`	Entry points
`notebooks/`	EDA & analysi

New Files (my contribution)

- `src/models/bdh.py` — BDH-GPU implementation
- `src/training/trainer.py` — unified config-driven training
- `src/evaluation/puzzle_evaluator.p` — ELO estimation
- `tests/test_bdh_model.py` — forward pass & shape tests
- `tests/test_training_smoke.py` — 1-batch smoke test

DevOps Toolchain

Environment: uv

```
uv sync
uv run python scripts/training/train.
  --config configs/train_bdh_small.ya
```

CI/CD (GitHub Actions)

- `ruff check` — linting
- `ruff format -check` — formatting
- `pytest tests/ -v` — all tests
- Triggered on push & PR to `main`

Testing (pytest)

- Data loading & encoding
- BDH forward pass & output shape
- Loss computation sanity check
- 1-batch training smoke test
- Evaluation pipeline test

Bonus Targets

- Docker container
- Streamlit demo
- Inference benchmarking
- W&B Sweeps

Reproducibility & Experiment Tracking

Reproducibility Guarantees:

Requirement	Implementation
Random seeds	YAML → `tf`, `np`, `random`
Fixed splits	Seeded shuffle, saved to JSON
Config-driven	1 YAML per experiment
Best model	`ModelCheckpoint`
Pinned deps	`uv.lock`

Unified Entry Point:
One script, any architecture:

```
uv run python scripts/training/train.
  --config configs/train_bdh_small.ya
```

Weights & Biases Integration:

- All hyperparams logged automatically
- Training curves (loss, MAE)
- ELO estimates per experiment
- Model artifacts saved

Planned Experiments ($\geq$7)

1. MLP baseline
2. ViT-Small baseline
3. BDH-Tiny (K=4)
4. BDH-Small (K=8)
5. BDH-Small depth ablation
6. BDH-Medium (K=12)
7. BDH + adaptive depth

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Compute Budget (100h A100)

Task	Hours	Cumulative	Phase
Data pipeline + encoding	2h	2h	Infrastructure
MLP baseline	2h	4h	Baselines
ViT-Small baseline	5h	9h	Baselines
BDH-Tiny (fast iteration)	5h	14h	BDH Development
BDH-Small (main model)	20h	34h	BDH Development
Depth ablation (K=4,8,12)	15h	49h	Ablations
BDH-Medium (scale-up)	40h	89h	Advanced
Evaluation + misc	5h	94h	Evaluation
Buffer	6h	100h	Safety

BDH-Medium is expendable — if behind schedule, reallocate to more ablations.

Meeting Roadmap

#	Meeting	Deliverable	Engineering Focus
3	Proposal	This presentation	—
4	Repo & CI	Public repo, CI green	`uv`, `ruff`, `pytest`
5	EDA + Baseline	Notebooks, MLP/ViT	Data pipeline tests
6	Reproducible	Config training, W&B	Seeds, checkpoints
7	First BDH	BDH-Tiny/Small results	W&B $\geq$3 experiments
8	Improvements	Tuning, depth ablation	Before/after comparison
9	Error Analysis	Failure by puzzle tier	Overfit/underfit analysis
10	Advanced	BDH-Medium, ablations	W&B $\geq$7 experiments
11	Final Review	Clean repo, README	CI green, all tests pass
12	Presentation	15–20 min final	Full results + conclusions

Risks & Mitigations

High Risk

BDH underperforms ViT

Mitigation: Still valid as architecture comparison study. Strong baselines ensure meaningful contribution regardless.

High Risk

No reference implementation

Mitigation: Start with BDH-Tiny, extensive unit tests, verify gradient flow early. *(Also: bonus for engineering quality.)*

Medium Risk

Compute overrun

Mitigation: Strict hour tracking. BDH-Medium is expendable.

Low Risk

Data pipeline issues

Mitigation: Reuse proven pipeline. HuggingFace `datasets` handles downloading.

Fallback Strategy

If BDH fails to converge → simplify (remove Hebbian updates, reduce to linear attention) → project becomes "Lessons Learned" with strong engineering.

Outline

1. Problem Definition
2. Dataset
3. Evaluation Metric
4. Architecture
5. Engineering Plan
6. Timeline, Risks & Compute
7. Summary

Summary

1. **Problem:** Searchless chess — predict position quality without search
   Supervised regression: FEN $\rightarrow$ score $\in [-1, 1]$
2. **Dataset:** 316M positions (Lichess-Stockfish-Normalized, HuggingFace)
   Proven to produce 1960 ELO models
3. **Architecture:** BDH-GPU — post-transformer with Hebbian state
   3 scales (1.6M–12.6M params), depth ablation $K \in \{4, 8, 12\}$
4. **Metric:** MSE (training) + ELO via 1200 puzzle benchmark
   Direct comparison to ViT-Small (1817 ELO)
5. **Engineering:** `uv` + CI/CD + `ruff` + `pytest` + W&B + YAML configs
   Config-driven, reproducible, $\geq$7 tracked experiments

Compute: 100h on NVIDIA A100

Thank You!

Questions?

Dataset: `huggingface.co/datasets/mateuszgrzyb/lichess-stockfish`

Reference repo: `github.com/mateuszgrzyb-pl/searchless-chess`

BDH paper: Available upon request

Input: $8 \times 8 \times 12$ board tensor $\xrightarrow{\text{BDH-GPU } (K \text{ passes})}$ Output: position score $\in (-1, 1)$

Backup: BDH-GPU Core Equations

Trainable parameters: $E, D_x, D_y \in \mathbb{R}^{n \times d}$
Total: $\approx (3 + o(1)) \cdot n \cdot d$

Per-layer recurrence (simplified):

1. **Update activations:** $\mathbf{x}_l \leftarrow \mathbf{x}_{l-1} + \mathrm{ReLU}(D_x \cdot \mathbf{v})$
2. **Read from fast state:** $\mathbf{a}_l = E \cdot \rho_{l-1} \cdot \mathbf{x}_l$
3. **Produce sparse output:** $\mathbf{y}_l = \mathrm{SparseGate}(\mathbf{a}_l)$
4. **Hebbian state update:** $\rho_l = \rho_{l-1} + \mathbf{v}_l^* \cdot \mathbf{x}_l^T$ where $\mathbf{v}_l^* = E^T \mathbf{y}_l$

Key property: Same weights at every layer $\rightarrow$ variable depth K is free.

Backup: YAML Config Example

```yaml
experiment:
  name: "bdh-small-k8"
  seed: 2001
  wandb_project: "bdh-searchless-chess"

model:
  architecture: "bdh"
  n_neurons: 8192
  d_rank: 256
  k_depth: 8
  dropout: 0.1

training:
  epochs: 100
  learning_rate: 1e-4
  min_learning_rate: 1e-6
  weight_decay: 1e-4
  batch_size: 512
  scheduler: "cosine"
  mixed_precision: true
```

Backup: ELO Estimation Method

1. **1,200 puzzles** across 12 difficulty tiers (399–3213 ELO)
2. For each tier: accuracy $= \frac{\text{correct puzzles}}{100}$
3. Fit linear regression: Rating $= f(\text{Accuracy})$
4. **ELO estimate** = predicted rating at 50% accuracy

Example from prior work (Grzyb, 2025):

Model	**Tier 4** (1000)	**Tier 6** (1500)	**Tier 8** (2000)
CNN (1.18M)	43%	22%	7%
ViT-Small (2.6M)	82%	68%	34%
ViT-Medium (9.5M)	88%	86%	45%